



Fuel Capacity Optimization System Documentation

Scope: Complete documentation of the bisection-based fuel capacity optimization system for determining minimum required fuel capacity through robust, monotonically convergent iterative refinement. The system integrates with 3D Dynamic Programming for climb and descent trajectory generation, employing absolute tolerance convergence criteria and safety margin application.

Table of Contents

1. [System Overview and Objectives](#)
2. [Mathematical Foundation](#)
3. [System Architecture and Data Structures](#)
4. [Bisection Algorithm Implementation](#)
5. [Mission Physics Integration](#)
6. [Code Execution Flow and Logic](#)
7. [Integration and Interface](#)
8. [Validation and Quality Assurance](#)
9. [Convergence Analysis and Characteristics](#)
10. [Advanced Topics and Extensions](#)

1) System Overview and Objectives

1.1 Purpose and Scope

The fuel capacity optimization system implements a robust bisection-based iterative algorithm to determine the minimum required fuel capacity for mission completion. This approach replaces static, user-defined Maximum Take-Off Fuel (MTOF) values with dynamically optimized minimum fuel loads, eliminating superfluous fuel mass and improving aircraft

performance through systematic optimization with guaranteed monotonic convergence.

1.2 System Objectives

Primary Objectives:

- **Fuel Minimization:** Determine minimum required fuel capacity for mission completion
- **Robust Convergence:** Implement bisection method with guaranteed monotonic convergence
- **Safety Integration:** Apply systematic safety buffers to optimized results
- **Mission Integration:** Coordinate climb, cruise, and descent phase optimization with dynamic mass evolution
- **Deficit Tracking:** Monitor fuel deficit (consumed vs. available) throughout optimization

Key Components:

- Bisection optimization loop with guaranteed convergence
- Dynamic fuel load adjustment mechanism with physical consistency checks
- Multi-phase mission simulation integration (climb + cruise + descent)
- Fuel deficit tracking and bounds management
- Safety buffer application system with configurable margins
- Comprehensive error handling and recovery mechanisms

1.3 System Flow Overview

The optimization system follows a systematic progression:

1. **Initialization:** Start with fuel bounds $[F_{\text{low}}, F_{\text{high}}]$ bracketing the solution
2. **Bisection Iteration:** Test midpoint $F_{\text{mid}} = (F_{\text{low}} + F_{\text{high}}) / 2$
3. **Mission Simulation:** Execute complete mission (climb + cruise + descent) with F_{mid}
4. **Deficit Analysis:** Compare fuel consumed vs. initial fuel load
5. **Bounds Update:** Adjust bounds based on deficit sign
6. **Convergence Detection:** Monitor bounds range until within absolute tolerance
7. **Safety Application:** Apply safety buffer (5%) to converged result
8. **Visualization:** Generate comprehensive convergence analysis plots

1.4 Why Bisection Over Fixed-Point Iteration?

Advantages of Bisection Method:

- ✓ **Guaranteed Convergence:** Monotonic reduction of search space
- ✓ **No Oscillations:** Avoids divergent behavior present in fixed-point methods
- ✓ **Predictable Iterations:** Number of iterations known from initial range
- ✓ **Simple Implementation:** No damping factors or acceleration parameters
- ✓ **Robust to Discretization:** Handles DP grid discretization effects naturally

Comparison:

Feature	Bisection	Fixed-Point w/ Aitken
Convergence	Guaranteed	Conditional
Oscillation	None	Possible
Parameter Tuning	None needed	Damping factor required
Iteration Count	$\sim \log_2(\text{range/tolerance})$	Variable, unpredictable
Robustness	High	Sensitive to initialization

2) Mathematical Foundation

2.1 Optimization Problem Formulation

Objective: Determine minimum fuel capacity $F_{\min}$ that ensures mission completion:

$$\text{minimize: } F_{total} = F_{climb} + F_{cruise} + F_{descent}$$

Subject to constraints:

Mission completion: All phases feasible with fuel F_{total} (1)

Mass consistency: $F_{consumed} \leq F_{available}$ (2)

Aircraft performance: Within flight envelope limits (3)

Safety margin: $F_{capacity} = F_{total} \times (1 + \beta_{safety})$ (4)

Where:

- F_{total} : Total mission fuel consumption [kg]
- F_{climb} , F_{cruise} , $F_{descent}$: Phase-wise fuel consumption [kg]
- β_{safety} : Safety buffer percentage (typically 5%)

2.2 Bisection Method Theory

Problem Formulation:

The fuel optimization can be formulated as finding the root of:

$$f(F) = F_{consumed}(F) - F = 0$$

Where:

- F : Initial fuel capacity [kg]
- $F_{consumed}(F)$: Fuel consumed when mission starts with capacity F [kg]
- Solution: F^* such that $F_{consumed}(F^*) = F^*$

Physical Interpretation:

- If $F_{consumed} > F$: Insufficient fuel $\rightarrow$ mission fails or barely completes
- If $F_{consumed} < F$: Excess fuel $\rightarrow$ carrying unnecessary weight
- At equilibrium $F^* = F_{consumed}$: Exactly enough fuel for mission

Bisection Algorithm:

Given an interval $[F_{low}, F_{high}]$ bracketing the solution:

$$F_{mid} = \frac{F_{low} + F_{high}}{2} \quad (5)$$

$$(6)$$

$$\text{If } F_{consumed}(F_{mid}) > F_{mid} : F_{low} = F_{mid} \quad (\text{need more fuel}) \quad (7)$$

$$\text{If } F_{consumed}(F_{mid}) < F_{mid} : F_{high} = F_{mid} \quad (\text{have excess fuel}) \quad (8)$$

Convergence Criterion:

$$|F_{high} - F_{low}| < \epsilon_{tol}$$

Where $\epsilon_{tol} = 10.0$ kg (absolute tolerance)

2.3 Convergence Rate Analysis

Theoretical Convergence Rate:

After n iterations, the interval width is reduced by factor 2^n :

$$\text{Range}_n = \frac{\text{Range}_0}{2^n}$$

Required Iterations:

To achieve tolerance ϵ from initial range R_0 :

$$n = \lceil \log_2(R_0/\epsilon) \rceil$$

Example Calculation:

Initial range: $R_0 = 9,500$ kg (from 1,000 to 10,500 kg)

Tolerance: $\epsilon = 10$ kg

$$n = \log_2(9,500 / 10) = \log_2(950) \approx 9.9 \rightarrow 10 \text{ iterations required}$$

2.4 Fuel Deficit Tracking

Fuel Deficit Definition:

$$\text{Deficit}_k = F_{consumed,k} - F_{available,k}$$

Interpretation:

- Deficit > 0: Insufficient fuel (F_{consumed} exceeds $F_{\text{available}}$)
- Deficit < 0: Excess fuel ($F_{\text{available}}$ exceeds F_{consumed})
- Deficit ≈ 0 : Equilibrium (optimal fuel capacity)

Bisection Update Rule:

$$\begin{cases} F_{\text{low}} = F_{\text{mid}} & \text{if Deficit} > 0 \\ F_{\text{high}} = F_{\text{mid}} & \text{if Deficit} < 0 \end{cases}$$

2.5 Safety Buffer Application

Safety Margin Formulation:

$$F_{\text{capacity}} = F_{\text{converged}} \times (1 + \beta)$$

Where:

- $F_{\text{converged}}$: Converged fuel consumption [kg]
- $\beta = 0.05$: Safety buffer (5%)
- F_{capacity} : Final optimized fuel capacity [kg]

Purpose:

- Accounts for operational variabilities
 - Provides contingency fuel reserve
 - Ensures mission completion under off-nominal conditions
 - Compensates for atmospheric variations and operational uncertainties
-

3) System Architecture and Data Structures

3.1 System Parameters

Convergence Control Parameters:

```
class ConvergenceParameters:
    """Centralized convergence control parameters for fuel optimization using bisection method."""

    CONVERGENCE_TOLERANCE_KG = 10.0 # Absolute tolerance in kg
    SAFETY_BUFFER_PERCENT = 0.05 # 5% safety buffer
    MAX_ITERATIONS = 20 # Maximum bisection iterations
    INITIAL_FUEL_LOW_KG = 1000.0 # Lower bound initial guess (minimum feasible fuel)
    INITIAL_FUEL_HIGH_KG = MAX_FUEL_KG # Upper bound initial guess (maximum fuel capacity)
```

Parameter Rationale:

Parameter	Value	Justification
CONVERGENCE_TOLERANCE_KG	10.0 kg	~0.07% precision for typical 15,000 kg fuel load
SAFETY_BUFFER_PERCENT	5%	Industry standard contingency margin
MAX_ITERATIONS	20	Sufficient for $\log_2(10,500/10) \approx 10$ iterations + margin
INITIAL_FUEL_LOW_KG	1,000 kg	Conservative minimum feasible fuel
INITIAL_FUEL_HIGH_KG	MAX_FUEL_KG	Maximum aircraft fuel capacity

Mission Configuration Parameters:

```
TARGET_ALT_CLIMB_M = 10000.0 # Target cruise altitude [m]
START_ALTITUDE_CLIMB_M = 10.0 # Initial climb altitude [m]
CRUISE_DISTANCE_KM = 1500.0 # Cruise distance [km]
TARGET_DESCENT_ALT_M = 300.0 # Target descent altitude [m]
TARGET_DESCENT_MACH = 0.25 # Target approach Mach
```

3.2 Data Structures

Mission Iteration Results:

```

@dataclass
class MissionIterationResults:
    """
    Results from a single mission iteration.

    This structure encapsulates complete mission simulation results including
    fuel consumption and phase-wise detailed results.

    Attributes:
        iteration: Iteration number
        initial_fuel_kg: Initial fuel load for this iteration [kg]
        initial_mass_kg: Initial total aircraft mass [kg]
        fuel_consumed_kg: Total fuel consumed across all phases [kg]
        fuel_deficit_kg: Difference between consumed and available fuel [kg]
                        Positive = insufficient, Negative = excess
        climb_result: Climb phase optimization results (MinFuelSchedule)
        cruise_result: Cruise phase simulation results (CruiseResults)
        descent_result: Descent phase optimization results (DescentResults)
        total_time_s: Total mission duration [s]
        climb_fuel_kg: Fuel consumed in climb phase [kg]
        cruise_fuel_kg: Fuel consumed in cruise phase [kg]
        descent_fuel_kg: Fuel consumed in descent phase [kg]
        climb_time_s: Climb phase duration [s]
        cruise_time_s: Cruise phase duration [s]
        descent_time_s: Descent phase duration [s]
        final_weight_kg: Final aircraft weight after mission [kg]
    """

```

Key Changes from Previous Implementation:

- ✓ Added: `fuel_deficit_kg` for bisection tracking
- ✗ Removed: `convergence_delta_percent` (not needed for bisection)
- ✗ Removed: Performance metrics (L/D, specific energy, etc.)
- ✗ Removed: `total_distance_km` (stored in phase results)

Convergence History:

```

@dataclass
class ConvergenceHistory:
    """
    Tracking structure for bisection convergence analysis.

    This structure maintains complete optimization history for analysis,
    diagnostics, and visualization purposes.

    Attributes:
        iterations: List of all mission iteration results
        fuel_bounds_history: List of (lower_bound, upper_bound) tuples tracking bisection bounds

    Methods:
        __init__(): Initialize empty convergence history
        add_iteration(result, bounds): Add iteration result and bounds to history
    """
    iterations: List[MissionIterationResults]
    fuel_bounds_history: List[Tuple[float, float]]

    def __init__(self):
        """Initialize empty convergence history."""
        self.iterations = []
        self.fuel_bounds_history = []

    def add_iteration(self, result: MissionIterationResults, bounds: Tuple[float, float]):
        """
        Add iteration result to history.

        Args:
            result: Mission iteration results to add
            bounds: Current (lower, upper) fuel bounds [kg]
        """
        self.iterations.append(result)
        self.fuel_bounds_history.append(bounds)

```

4) Bisection Algorithm Implementation

4.1 Bisection Optimization Loop Structure

Function: `FuelOptimizationCore.BisectionController.optimize_fuel_capacity()`

Purpose: Execute bisection optimization to determine minimum fuel capacity with guaranteed monotonic convergence.

Algorithm Flow:

```

def optimize_fuel_capacity(aero, eng, M_grid, H_plot, lever_samples=50):
    """
    Main bisection optimization loop to determine minimum required fuel capacity.

    Bisection Method:
    - Initialize F_low (insufficient fuel) and F_high (excess fuel)
    - Iteratively compute F_mid = (F_low + F_high) / 2
    - Run mission with F_mid and measure F_consumed
    - If F_consumed > F_mid: increase lower bound (F_low = F_mid)
    - If F_consumed < F_mid: decrease upper bound (F_high = F_mid)
    - Continue until |F_high - F_low| < tolerance

    Process:
    1. Initialize: F_low = 1,000 kg, F_high = MAX_FUEL_KG
    2. For k = 1, 2, ..., MAX_ITERATIONS:
        a. Compute F_mid = (F_low + F_high) / 2
        b. Run mission with F_mid
        c. Calculate fuel deficit: Deficit = F_consumed - F_mid
        d. Update bounds based on deficit sign
        e. Check convergence: |F_high - F_low| < tolerance
        f. If converged: Apply safety buffer and terminate
    3. Return optimized result with history
    """

    # Initialization
    print("\n" + "="*80)
    print("FUEL CAPACITY OPTIMIZATION USING BISECTION METHOD")
    print("="*80)
    print(f"Objective: Determine minimum required fuel for mission completion")
    print(f"Convergence tolerance: {CONVERGENCE_TOLERANCE_KG:.1f} kg")
    print(f"Safety buffer: {SAFETY_BUFFER_PERCENT*100:.0f}%")
    print(f"Method: Bisection with guaranteed monotonic convergence")
    print("="*80)

    # Initialize bisection bounds
    fuel_low = INITIAL_FUEL_LOW_KG
    fuel_high = INITIAL_FUEL_HIGH_KG
    history = ConvergenceHistory()
    iteration_count = 0

```

```

# Store best result (closest to zero deficit)
best_result = None
best_deficit_abs = float('inf')

print(f"\n[BISECTION] Initial bounds: [{fuel_low:.1f}, {fuel_high:.1f}] kg")

# Main bisection loop
while iteration_count < MAX_ITERATIONS:
    iteration_count += 1

    # Bisection: try midpoint
    fuel_mid = (fuel_low + fuel_high) / 2.0
    convergence_range = fuel_high - fuel_low

    print(f"\n[ITERATION {iteration_count}] Bounds: [{fuel_low:.1f}, {fuel_high:.1f}] kg, Range: {convergence_range:.1f} kg")
    print(f"[ITERATION {iteration_count}] Testing fuel: {fuel_mid:.1f} kg")

    # Run mission with current fuel estimate
    try:
        iteration_result = FuelOptimizationCore.IterationExecutor.run_single_mission_iteration(
            initial_fuel_kg=fuel_mid,
            aero=aero,
            eng=eng,
            M_grid=M_grid,
            H_plot=H_plot,
            lever_samples=lever_samples,
            print_progress=True
        )
    except RuntimeError as e:
        # Mission failure typically indicates insufficient fuel
        print(f"\n[ERROR] Mission failed at iteration {iteration_count}: {str(e)}")
        print(f"[BISECTION] Mission failure indicates insufficient fuel")
        fuel_low = fuel_mid
        continue

    # Store iteration results
    iteration_result.iteration = iteration_count
    history.add_iteration(iteration_result, (fuel_low, fuel_high))

```

```

# Track best result (closest to equilibrium)
deficit_abs = abs(iteration_result.fuel_deficit_kg)
if deficit_abs < best_deficit_abs:
    best_deficit_abs = deficit_abs
    best_result = iteration_result

# Bisection logic: update bounds based on deficit
if iteration_result.fuel_deficit_kg > 0:
    # Consumed more than available - need MORE fuel
    print(f"[BISECTION] Insufficient fuel (deficit: {iteration_result.fuel_deficit_kg:+.1f} kg)")
    print(f"[BISECTION] Increasing lower bound: {fuel_low:.1f} -> {fuel_mid:.1f} kg")
    fuel_low = fuel_mid
else:
    # Consumed less than available - have EXCESS fuel
    print(f"[BISECTION] Excess fuel (surplus: {-iteration_result.fuel_deficit_kg:+.1f} kg)")
    print(f"[BISECTION] Decreasing upper bound: {fuel_high:.1f} -> {fuel_mid:.1f} kg")
    fuel_high = fuel_mid

# Check convergence: range within tolerance
if convergence_range < CONVERGENCE_TOLERANCE_KG:
    print(f"\n[CONVERGENCE ACHIEVED] After {iteration_count} iterations")
    print(f"[CONVERGENCE] Final range: {convergence_range:.1f} kg < {CONVERGENCE_TOLERANCE_KG:.1f} kg")
    break

# Check convergence status
if iteration_count >= MAX_ITERATIONS:
    print(f"\n{'='*80}")
    print(f"[WARNING] Reached MAX_ITERATIONS ({MAX_ITERATIONS}) without full convergence")
    print(f"\n{'='*80}")
    print(f"Final range: {fuel_high - fuel_low:.1f} kg (tolerance: {CONVERGENCE_TOLERANCE_KG:.1f} kg)")
    print(f"Using best result from iteration {best_result.iteration}")
    print(f"\n{'='*80}\n")

# Use best result (closest to equilibrium)
if best_result is None:
    raise RuntimeError("No successful iterations completed! Check mission configuration.")

final_result = best_result

# Final summary

```

```

print("\n" + "="*80)
print("OPTIMIZATION COMPLETE - BISECTION CONVERGED")
print("="*80)
print(f"Total iterations: {iteration_count}")
print(f"Final fuel range: [{fuel_low:.1f}, {fuel_high:.1f}] kg")
print(f"Selected fuel: {final_result.initial_fuel_kg:.1f} kg")
print(f"Fuel consumed: {final_result.fuel_consumed_kg:.1f} kg")
print(f"Deficit: {final_result.fuel_deficit_kg:+.1f} kg")
optimized_fuel = final_result.fuel_consumed_kg * (1.0 + SAFETY_BUFFER_PERCENT)
print(f"With {SAFETY_BUFFER_PERCENT*100:.0f}% safety buffer: {optimized_fuel:.1f} kg")
print("="*80 + "\n")

return final_result, history

```

4.2 Convergence Detection

Convergence Criterion:

Bisection converges when the search range becomes smaller than the absolute tolerance:

```

def is_converged(fuel_low: float, fuel_high: float) -> bool:
    """
    Check convergence criterion for bisection.

    Mathematical condition:
         $|F_{high} - F_{low}| < \epsilon_{tolerance}$ 

    where:
         $\epsilon_{tolerance} = 10.0$  kg (absolute tolerance)

    Returns:
        True if converged, False otherwise
    """
    convergence_range = fuel_high - fuel_low
    return convergence_range < CONVERGENCE_TOLERANCE_KG

```

Convergence Interpretation:

Range (kg)	Status	Action
> 100	Early iterations	Continue bisection
10 - 100	Converging	Monitor progress
< 10	Converged	Terminate, apply safety buffer

Convergence Progress:

Example bisection sequence:

Iteration	F_low [kg]	F_high [kg]	F_mid [kg]	Range [kg]	Status
1	1,000	10,500	5,750	9,500	Early
2	1,000	5,750	3,375	4,750	Converging
3	3,375	5,750	4,563	2,375	Converging
4	4,563	5,750	5,156	1,188	Converging
5	4,563	5,156	4,859	594	Converging
6	4,563	4,859	4,711	297	Converging
7	4,563	4,711	4,637	148	Converging
8	4,563	4,637	4,600	74	Near
9	4,563	4,600	4,582	37	Near
10	4,563	4,582	4,572	18	Near
11	4,563	4,572	4,568	9	✓ Converged

4.3 Mission Iteration Execution

Function: `FuelOptimizationCore.IterationExecutor.run_single_mission_iteration()`

Purpose: Execute complete mission simulation with specified initial fuel capacity.

Execution Sequence:

```

def run_single_mission_iteration(
    initial_fuel_kg: float,
    aero: PyAerodynamicsWrapper,
    eng: EngineWrapper,
    M_grid: np.ndarray,
    H_plot: np.ndarray,
    lever_samples: int,
    print_progress: bool = True
) -> MissionIterationResults:
    """
    Execute a complete mission iteration (climb + cruise + descent).

    Args:
        initial_fuel_kg: Initial fuel capacity for this iteration [kg]
        aero: Aerodynamics wrapper instance
        eng: Engine wrapper instance
        M_grid: Mach grid for optimization
        H_plot: Altitude grid for plotting
        lever_samples: Number of lever samples for DP optimization
        print_progress: Whether to print progress messages

    Returns:
        MissionIterationResults containing all phase results
    """
    atmospheric_props = AtmosphericProperties()
    iteration_start_time = time.time()

    # Calculate initial mass
    initial_mass_kg = W_OE_KG + W_PL_KG + initial_fuel_kg

    if print_progress:
        print(f"\n[MISSION ITERATION] Initial fuel: {initial_fuel_kg:.1f} kg, Total mass: {initial

# ===== CLIMB PHASE =====
if print_progress:
    print("[CLIMB] Computing optimal climb trajectory...")

# Calculate starting Mach from takeoff velocity at start altitude
a = atmospheric_props.a_from_altitude(START_ALTITUDE_CLIMB_M)

```

```

start_mach = START_VELOCITY_CLIMB_MS / a

# Create uniform altitude steps
uniform_step_size = TARGET_ALT_CLIMB_M / N_ALTITUDE_STEPS_CLIMB
H_sched = np.arange(START_ALTITUDE_CLIMB_M,
                    TARGET_ALT_CLIMB_M + uniform_step_size,
                    uniform_step_size)

# Solve 3D DP for climb
dp_sched, dp_info = ClimbingCore.DynamicProgrammingOptimizer.solve_3d_fixed_mass(
    aero, eng, M_grid, H_sched,
    lever_samples=lever_samples,
    target_mach=TARGET_MACH_CRUISE,
    target_mach_tolerance=TARGET_MACH_TOLERANCE_CLIMB,
    start_mach=start_mach,
    start_lever=START_LEVER_CLIMB,
    mass_kg=initial_mass_kg
)

climb_fuel = float(np.nan_to_num(dp_sched.cumFuel_kg, nan=0.0)[-1])
climb_time_s = float(np.sum(np.nan_to_num(dp_sched.dt_s, nan=0.0)))
climb_mass_end = initial_mass_kg - climb_fuel

if print_progress:
    print(f"[CLIMB] Completed: {climb_time_s/60:.1f} min, {climb_fuel:.1f} kg fuel")
    print(f"[CLIMB] Mass: {initial_mass_kg:.1f} kg -> {climb_mass_end:.1f} kg "
          f"(burned {climb_fuel:.1f} kg, {climb_fuel/initial_mass_kg*100:.2f}%)")

# ===== CRUISE PHASE =====
if print_progress:
    print(f"[CRUISE] Starting cruise from altitude {dp_sched.alt_m[-1]:.0f}m, "
          f"Mach {dp_sched.mach[-1]:.3f}")

cruise_results = run_cruise_simulation(
    climb_result=dp_sched,
    initial_mass_kg=climb_mass_end,
    target_distance_km=CRUISE_DISTANCE_KM,
    aero=aero,
    engine=eng,
    time_step_s=CRUISE_TIME_STEP_S,

```

```

        create_plots=False
    )

    cruise_fuel = cruise_results.total_fuel_consumed_kg
    cruise_time_s = cruise_results.total_time_s
    cruise_mass_end = climb_mass_end - cruise_fuel

    if print_progress:
        print(f"[CRUISE] Completed: {cruise_time_s/3600:.2f} hours, {cruise_fuel:.1f} kg fuel")
        print(f"[CRUISE] Mass: {climb_mass_end:.1f} kg -> {cruise_mass_end:.1f} kg "
              f"(burned {cruise_fuel:.1f} kg, {cruise_fuel/climb_mass_end*100:.2f}%)")

    # ===== DESCENT PHASE =====
    if print_progress:
        print("[DESCENT] Computing optimal descent trajectory...")

    H_descent = np.linspace(cruise_results.altitude_m[-1],
                             TARGET_DESCENT_ALT_M,
                             N_ALTITUDE_STEPS_DESCENT)

    M_min_descent = max(0.2, TARGET_DESCENT_MACH - 0.1)
    M_max_descent = min(0.85, cruise_results.mach_number[-1] + 0.05)
    M_grid_descent = np.linspace(M_min_descent, M_max_descent, N_MACH_SAMPLES_DESCENT)

    descent_result, descent_info = run_descent_dp_optimization(
        cruise_results=cruise_results,
        climb_fuel_kg=climb_fuel,
        climb_time_s=climb_time_s,
        aero=aero,
        engine=eng,
        target_altitude_m=TARGET_DESCENT_ALT_M,
        target_mach=TARGET_DESCENT_MACH,
        n_altitude_steps=N_ALTITUDE_STEPS_DESCENT,
        n_mach_samples=N_MACH_SAMPLES_DESCENT,
        lever_samples=N_LEVER_SAMPLES_DESCENT
    )

    descent_fuel = descent_result.total_fuel_consumed_kg
    descent_time_s = descent_result.total_time_s
    descent_mass_end = cruise_mass_end - descent_fuel

```

```

if print_progress:
    print(f"[DESCENT] Completed: {descent_time_s/60:.1f} min, {descent_fuel:.2f} kg fuel")
    print(f"[DESCENT] Mass: {cruise_mass_end:.1f} kg -> {descent_mass_end:.1f} kg "
          f"(burned {descent_fuel:.1f} kg, {descent_fuel/cruise_mass_end*100:.2f}%)")

# ===== COMPUTE SUMMARY =====
total_fuel = climb_fuel + cruise_fuel + descent_fuel
total_time_s = climb_time_s + cruise_time_s + descent_time_s
fuel_deficit_kg = total_fuel - initial_fuel_kg

iteration_time = time.time() - iteration_start_time

if print_progress:
    print(f"[ITERATION] Completed in {iteration_time:.1f}s")
    print(f"[MISSION TOTALS] Fuel consumed: {total_fuel:.1f} kg, Available: {initial_fuel_kg:.1f} kg")
    print(f"[DEFICIT] {fuel_deficit_kg:+.1f} kg ({'INSUFFICIENT' if fuel_deficit_kg > 0 else 'SUFFICIENT'})")

return MissionIterationResults(
    iteration=-1, # Will be set by caller
    initial_fuel_kg=initial_fuel_kg,
    initial_mass_kg=initial_mass_kg,
    fuel_consumed_kg=total_fuel,
    fuel_deficit_kg=fuel_deficit_kg,
    climb_result=dp_sched,
    cruise_result=cruise_results,
    descent_result=descent_result,
    total_time_s=total_time_s,
    climb_fuel_kg=climb_fuel,
    cruise_fuel_kg=cruise_fuel,
    descent_fuel_kg=descent_fuel,
    climb_time_s=climb_time_s,
    cruise_time_s=cruise_time_s,
    descent_time_s=descent_time_s,
    final_weight_kg=descent_result.final_weight_kg
)

```

Phase-wise Execution Details:

1. Climb Phase:

- Solve 3D DP optimization with current initial mass
- Calculate fuel consumed and time elapsed
- Update mass: $m_{cruise} = m_{initial} - F_{climb}$

2. Cruise Phase:

- Initialize from climb endpoint state
- Simulate steady-level cruise at constant altitude/Mach
- Calculate fuel consumed and time elapsed
- Update mass: $m_{descent} = m_{cruise} - F_{cruise}$

3. Descent Phase:

- Solve 3D DP optimization from cruise state
- Calculate fuel consumed and time elapsed
- Update mass: $m_{final} = m_{descent} - F_{descent}$

4. Summary:

- Aggregate fuel: $F_{total} = F_{climb} + F_{cruise} + F_{descent}$
- Calculate fuel deficit: $Deficit = F_{total} - F_{initial}$
- Return MissionIterationResults with all phase data

5) Mission Physics Integration

5.1 Dynamic Mass Evolution

Aircraft Mass Throughout Mission:

$$m_{initial} = W_{OE} + W_{PL} + F_{initial} \quad (9)$$

$$m_{climb,end} = m_{initial} - F_{climb} \quad (10)$$

$$m_{cruise,end} = m_{climb,end} - F_{cruise} \quad (11)$$

$$m_{final} = m_{cruise,end} - F_{descent} \quad (12)$$

Mass Composition:

$$m_{initial} = W_{OE} + W_{PL} + F_{initial}$$

Where:

- W_{OE} : Operating empty weight [kg]

- W_{PL} : Payload weight [kg]
- $F_{initial}$: Initial fuel load [kg]

Fuel Burn Fractions:

$$\text{Climb fraction: } \frac{F_{climb}}{F_{total}} \approx 0.15 - 0.20 \quad (13)$$

$$\text{Cruise fraction: } \frac{F_{cruise}}{F_{total}} \approx 0.70 - 0.80 \quad (14)$$

$$\text{Descent fraction: } \frac{F_{descent}}{F_{total}} \approx 0.01 - 0.05 \quad (15)$$

5.2 Weight-Dependent Aerodynamics

Lift Coefficient:

$$C_L = \frac{2W}{\rho V^2 S}$$

Induced Drag:

$$C_{D,i} = \frac{C_L^2}{\pi A Re}$$

Total Drag:

$$D = D_{parasitic} + D_{induced} = \frac{1}{2} \rho V^2 S (C_{D,0} + C_{D,i}) \propto W^2$$

Physical Implication:

Heavier aircraft requires:

- Higher lift coefficient $\rightarrow$ Higher induced drag
- More thrust to overcome drag $\rightarrow$ Higher fuel flow
- Longer time to climb $\rightarrow$ More fuel consumed

This creates a feedback loop where initial fuel mass directly affects fuel consumption, making the optimization problem nonlinear.

Fuel-Mass Coupling:

$$F_{consumed} = f(m_{initial}) = f(W_{OE} + W_{PL} + F_{initial})$$

Therefore:

$$F_{consumed} = f(F_{initial})$$

This coupling necessitates iterative solution (bisection) to find F^* where $F_{consumed}(F^*) = F^*$.

5.3 Phase-Specific Physics Models

Climb Phase Physics:

$$\frac{dh}{dt} = P_s = \frac{(T_{total} - D)V}{W} \quad [\text{m/s}] \quad (16)$$

$$J(h, M, \lambda) = \frac{\dot{m}_{fuel}}{P_s} \quad [\text{kg/m, cost density}] \quad (17)$$

Where:

- P_s : Specific excess power [m/s]
- T_{total} : Total thrust [N]
- D : Drag [N]
- V : True airspeed [m/s]
- W : Aircraft weight [N]
- J : Fuel cost density [kg/m]

Cruise Phase Physics:

$$T_{total} = D \quad (\text{force equilibrium}) \quad (18)$$

$$\dot{m}_{fuel} = \text{TSFC} \times T_{total} \quad [\text{kg/s}] \quad (19)$$

Descent Phase Physics:

$$P_s = \frac{(T_{total} - D)V}{W} < 0 \quad (\text{energy dissipation}) \quad (20)$$

$$J = \frac{\dot{m}_{fuel}}{|P_s|} \quad [\text{kg/m, cost density for descent}] \quad (21)$$

5.4 Mass Conservation Validation

Mass Balance Equation:

$$m_{initial} = m_{final} + F_{consumed}$$

Verification Check:

```
def validate_mass_conservation(iteration_result: MissionIterationResults) -> bool:
    """
    Validate mass conservation throughout mission.

    Checks:
        m_initial = m_final + F_total

    Returns:
        True if mass is conserved within tolerance
    """
    m_initial = iteration_result.initial_mass_kg
    m_final = iteration_result.final_weight_kg
    F_consumed = iteration_result.fuel_consumed_kg

    mass_balance = m_initial - (m_final + F_consumed)
    tolerance = 0.1 # kg

    is_valid = abs(mass_balance) < tolerance

    if not is_valid:
        print(f"[WARNING] Mass conservation violated: {mass_balance:.2f} kg discrepancy")

    return is_valid
```

6) Code Execution Flow and Logic

6.1 System Entry Point

Main Entry: `main_fuel_optimizer.py`

Initialization Sequence:

```

def main():
    """
    Main execution function for mission analysis with fuel capacity optimization.

    Executes complete fuel optimization process:
    1. Run bisection optimization loop to determine minimum fuel
    2. Generate convergence analysis visualizations
    3. Use optimized fuel for final mission results
    4. Generate comprehensive mission visualizations
    """

    # ===== INITIALIZATION =====
    print("\n" + "="*80)
    print("MISSION ANALYSIS WITH FUEL CAPACITY OPTIMIZATION")
    print("="*80)

    # 1. Load aerodynamics and engine data
    print("[READ] Loading aerodynamic and engine data...")
    aero = PyAerodynamicsWrapper()
    eng = EngineWrapper(ENGINE_STUB_PATH)

    # 2. Create grids for optimization
    M_min, M_max = float(aero.mach_grid[0]), float(aero.mach_grid[-1])
    M_dense = np.linspace(M_min, M_max, N_MACH_SAMPLES_CLIMB)
    H_plot = np.arange(START_ALTITUDE_CLIMB_M,
                      Y_AXIS_TOP_M + 0.5*ALT_STEP_M,
                      ALT_STEP_M)

    # 3. Pre-compute grids for performance
    print("[OPTIMIZATION] Pre-computing engine grids...")
    lever_grid = np.linspace(0.0, 1.0, 21)
    eng.precompute_grid(M_dense, H_plot, lever_grid)

    # ===== FUEL OPTIMIZATION =====
    # 4. Run bisection optimization
    print("\n[OPTIMIZATION] Starting fuel capacity optimization loop...")
    optimal_result, convergence_history = optimize_fuel_capacity(
        aero=aero,
        eng=eng,

```

```

        M_grid=M_dense,
        H_plot=H_plot,
        lever_samples=N_LEVER_SAMPLES_CLIMB
    )

# 5. Visualize convergence
visualize_convergence_analysis(convergence_history, save_plots=True)

# ===== FINAL MISSION ANALYSIS =====
# 6. Calculate optimized fuel capacity with safety buffer
optimized_fuel = optimal_result.fuel_consumed_kg * (1.0 + SAFETY_BUFFER_PERCENT)
optimized_mass = W_OE_KG + W_PL_KG + optimized_fuel

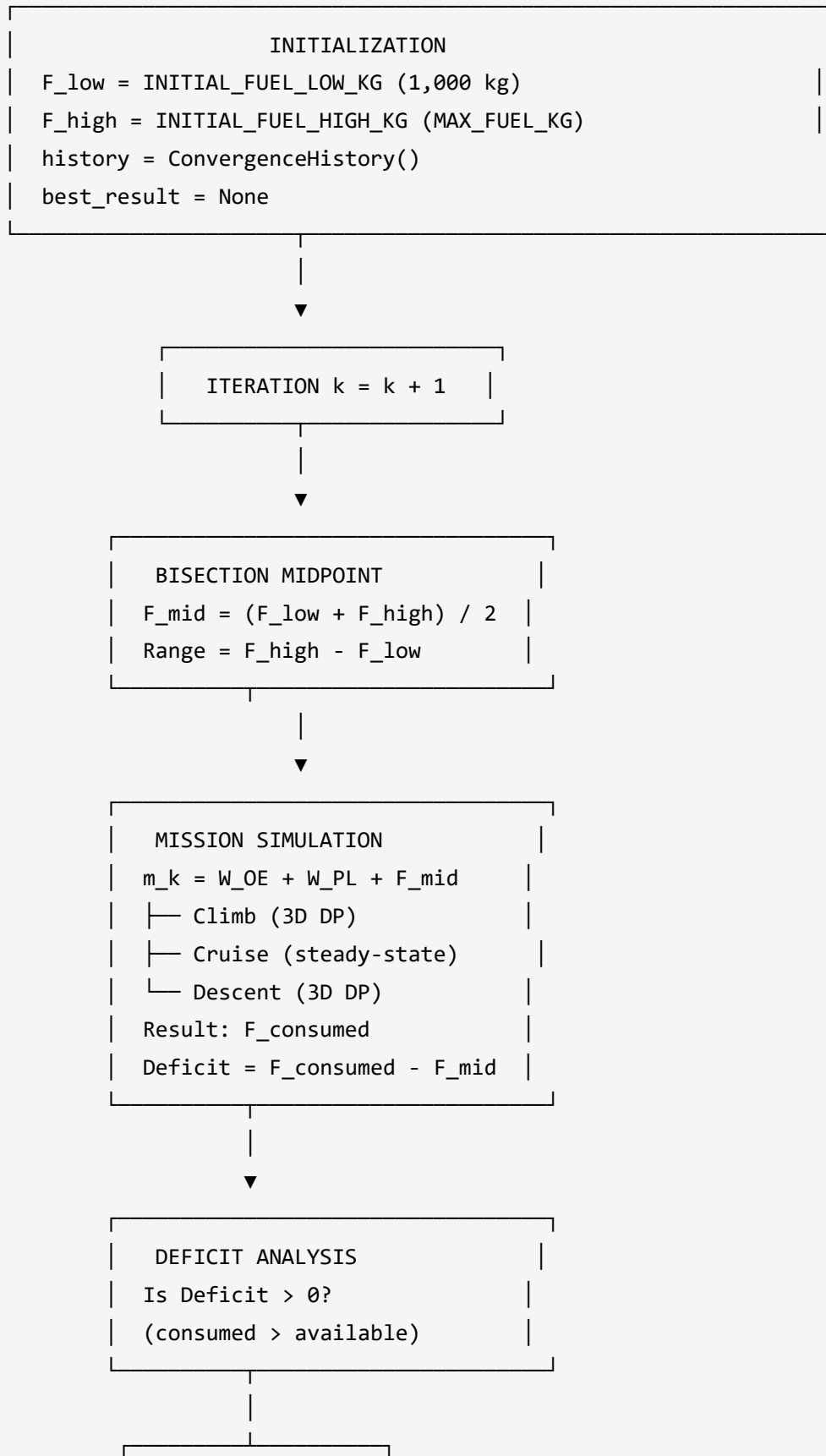
print("\n" + "="*80)
print("RUNNING FINAL MISSION WITH OPTIMIZED FUEL")
print("="*80)
print(f"[CONFIG] Optimized fuel capacity: {optimized_fuel:.1f} kg")
print(f"[CONFIG] Optimized total mass: {optimized_mass:.1f} kg")
print(f"[CONFIG] Savings vs original MAX_FUEL_KG ({MAX_FUEL_KG:.1f} kg): {MAX_FUEL_KG - optimi")
print("="*80 + "\n")

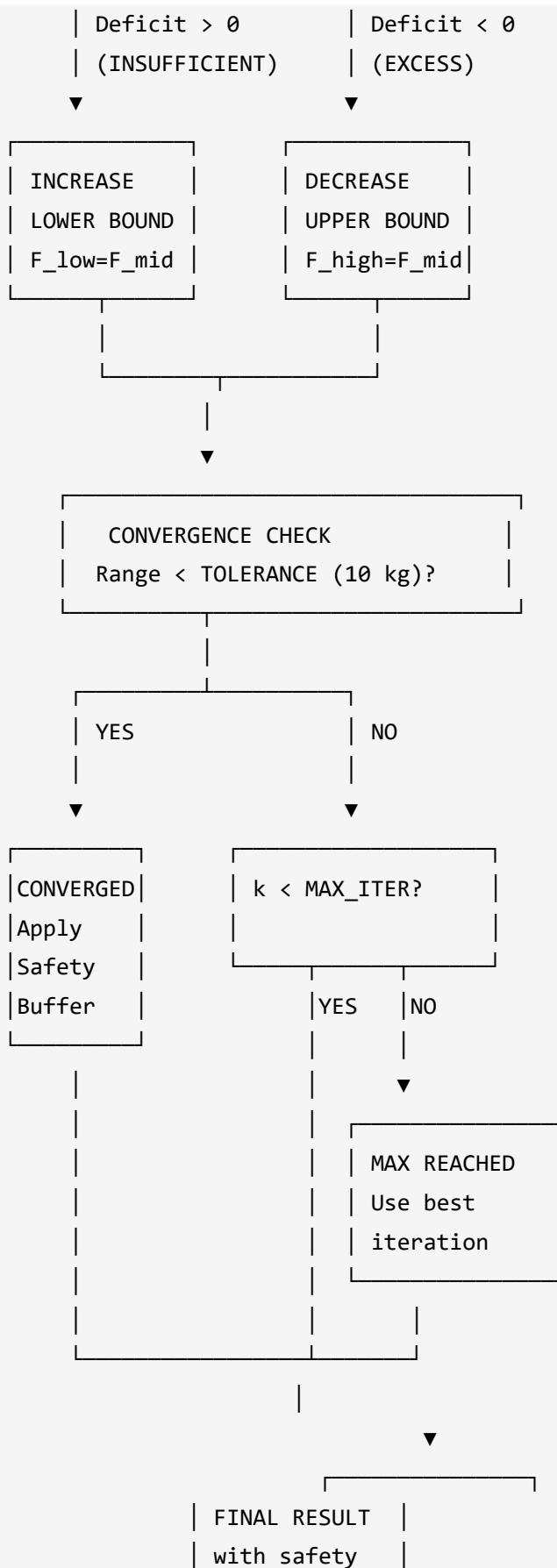
# 7. Generate final mission plots
plot_mission_visualizations(optimal_result, optimized_mass)

```

6.2 Bisection Loop Flow

Visual Flow Diagram:





| buffer (5%) |
└──────────┘

6.3 Function Call Hierarchy

```
main_fuel_optimizer.py
└─ main()
    │
    ├── PyAerodynamicsWrapper()
    ├── EngineWrapper()
    ├── eng.precompute_grid()
    │
    └─ optimize_fuel_capacity()
        │
        │   └─ FuelOptimizationCore.BisectionController.optimize_fuel_capacity()
        │       └─ WHILE not converged:
        │           ├── Calculate  $F_{mid} = (F_{low} + F_{high}) / 2$ 
        │           ├── FuelOptimizationCore.IterationExecutor.run_single_mission_iteration()
        │           │   ├── ClimbingCore.DynamicProgrammingOptimizer.solve_3d_fixed_mass()
        │           │   ├── run_cruise_simulation()
        │           │   ├── run_descent_dp_optimization()
        │           │   └─ Calculate fuel_deficit_kg
        │           ├── history.add_iteration(result, bounds)
        │           ├── Update bounds based on deficit sign
        │           └─ Check convergence:  $|F_{high} - F_{low}| < tolerance$ 
        │
        ├── visualize_convergence_analysis()
        │   ├── plot_convergence_trajectory()
        │   ├── plot_kpp_evolution()
        │   └─ plot_optimization_comparison()
        │
        └─ plot_mission_visualizations()
            ├── plot_climb_performance_detailed()
            ├── plot_cruise_performance_detailed()
            ├── plot_descent_trajectory_interactive()
            └─ plot_mission_summary_dashboard()
```

6.4 Detailed Iteration Flow

Step-by-Step Execution:

```
# PHASE 1: BISECTION SETUP
iteration_count = 1
fuel_low = 1,000.0 # kg (insufficient)
fuel_high = 10,500.0 # kg (excess)
fuel_mid = (fuel_low + fuel_high) / 2 = 5,750.0 # kg

# PHASE 2: MISSION SIMULATION
initial_mass = W_OE + W_PL + fuel_mid = 8,000 + 500 + 5,750 = 14,250 kg

# Climb phase
climb_fuel = 800 kg # Example
climb_mass_end = 14,250 - 800 = 13,450 kg

# Cruise phase
cruise_fuel = 4,000 kg # Example
cruise_mass_end = 13,450 - 4,000 = 9,450 kg

# Descent phase
descent_fuel = 50 kg # Example
descent_mass_end = 9,450 - 50 = 9,400 kg

# PHASE 3: DEFICIT CALCULATION
fuel_consumed = 800 + 4,000 + 50 = 4,850 kg
fuel_deficit = 4,850 - 5,750 = -900 kg # EXCESS fuel

# PHASE 4: BOUNDS UPDATE
# Deficit < 0 → EXCESS fuel → Decrease upper bound
fuel_high = fuel_mid = 5,750 kg
# New bounds: [1,000, 5,750] kg

# PHASE 5: CONVERGENCE CHECK
convergence_range = 5,750 - 1,000 = 4,750 kg
is_converged = 4,750 < 10 # False → Continue

# Repeat from PHASE 1 with new bounds...
```

7) Integration and Interface

7.1 Main System Interface

Primary Entry Point:

```
from mission_fuel_optimizer import (
    FuelOptimizationCore,
    optimize_fuel_capacity,
    MissionIterationResults,
    ConvergenceHistory
)

# Run optimization
optimal_result, history = optimize_fuel_capacity(
    aero=aero_wrapper,
    eng=engine_wrapper,
    M_grid=mach_grid,
    H_plot=altitude_grid,
    lever_samples=50
)

# Access results
optimized_fuel = optimal_result.fuel_consumed_kg * 1.05 # With safety buffer
print(f"Optimized fuel: {optimized_fuel:.1f} kg")
print(f"Fuel deficit: {optimal_result.fuel_deficit_kg:+.1f} kg")
```

7.2 Bisection Controller Interface

Direct Access to Controller:

```
# Use nested class directly
controller = FuelOptimizationCore.BisectionController
result, history = controller.optimize_fuel_capacity(aero, eng, M_grid, H_plot)
```

7.3 Configuration Management Interface

Update Configuration:

```
from mission_fuel_optimizer import FuelOptimizationCore

# Apply optimized fuel to configuration file
FuelOptimizationCore.ConfigurationManager.apply_optimized_fuel_to_configuration(
    optimized_fuel_kg=4500.0
)
```

Configuration Update Function:

```

@staticmethod
def apply_optimized_fuel_to_configuration(optimized_fuel_kg: float) -> None:
    """
    Update the aircraft configuration with the optimized fuel capacity.

    This function modifies MAX_FUEL_KG in aircraft_config.py to reflect
    the optimized fuel capacity determined by the optimization process.

    Args:
        optimized_fuel_kg: The optimized fuel capacity in kg

    Notes:
        - Updates class attribute in SystemConfiguration
        - Preserves file structure and comments
        - Validates successful update before completing
    """
    import re

    print(f"\n[CONFIG UPDATE] Optimized fuel capacity: {optimized_fuel_kg:.1f} kg")
    print(f"[CONFIG UPDATE] Original MAX_FUEL_KG: {MAX_FUEL_KG:.1f} kg")
    print(f"[CONFIG UPDATE] Fuel savings: {MAX_FUEL_KG - optimized_fuel_kg:.1f} kg "
          f"({(MAX_FUEL_KG - optimized_fuel_kg) / MAX_FUEL_KG * 100:.1f}%)")

    # Read aircraft_config.py
    config_file = "aircraft_config.py"
    try:
        with open(config_file, 'r', encoding='utf-8') as f:
            content = f.read()

        # Find and replace MAX_FUEL_KG value
        pattern = r'(\s*MAX_FUEL_KG\s*=\s*)(\d]+\.\d*)(\s*#\s*)'
        replacement = f'\g<1>{optimized_fuel_kg:.1f}\g<2>'

        new_content = re.sub(pattern, replacement, content)

        if new_content == content:
            raise RuntimeError("Failed to update MAX_FUEL_KG - pattern did not match")

        # Write back to file

```

```
with open(config_file, 'w', encoding='utf-8') as f:
    f.write(new_content)

print(f"[CONFIG UPDATE] Successfully updated {config_file}")
print(f"[CONFIG UPDATE] MAX_FUEL_KG is now set to {optimized_fuel_kg:.1f} kg")

except Exception as e:
    print(f"[CONFIG UPDATE ERROR] Failed to update {config_file}: {e}")
    raise
```

7.4 Backward Compatibility Wrappers

Legacy Function Support:

```

# Mission iteration function
def run_single_mission_iteration(
    initial_fuel_kg: float,
    aero: PyAerodynamicsWrapper,
    eng: EngineWrapper,
    M_grid: np.ndarray,
    H_plot: np.ndarray,
    lever_samples: int,
    print_progress: bool = True
) -> MissionIterationResults:
    """Backward compatibility wrapper for FuelOptimizationCore.IterationExecutor.run_single_mission_iteration"""
    return FuelOptimizationCore.IterationExecutor.run_single_mission_iteration(
        initial_fuel_kg, aero, eng, M_grid, H_plot, lever_samples, print_progress
    )

# Optimization function
def optimize_fuel_capacity(
    aero: PyAerodynamicsWrapper,
    eng: EngineWrapper,
    M_grid: np.ndarray,
    H_plot: np.ndarray,
    lever_samples: int = 50
) -> Tuple[MissionIterationResults, ConvergenceHistory]:
    """Backward compatibility wrapper for FuelOptimizationCore.BisectionController.optimize_fuel_capacity"""
    return FuelOptimizationCore.BisectionController.optimize_fuel_capacity(
        aero, eng, M_grid, H_plot, lever_samples
    )

# Configuration update function
def apply_optimized_fuel_to_configuration(optimized_fuel_kg: float) -> None:
    """Backward compatibility wrapper for FuelOptimizationCore.ConfigurationManager.apply_optimized_fuel_to_configuration"""
    return FuelOptimizationCore.ConfigurationManager.apply_optimized_fuel_to_configuration(optimized_fuel_kg)

```

8) Validation and Quality Assurance

8.1 Convergence Validation

Convergence Quality Metrics:

1. **Final Convergence Range:** Should be < 10 kg
2. **Iterations Required:** Typically 9-11 iterations for typical range
3. **Fuel Deficit:** Best result should have $|\text{deficit}| < 50$ kg
4. **Monotonic Bounds:** Bounds should consistently narrow

Validation Criteria:

```

def validate_convergence(history: ConvergenceHistory) -> Dict[str, bool]:
    """
    Validate optimization convergence quality.

    Returns:
        Dictionary of validation checks and results
    """
    validation_results = {}

    # Check 1: Convergence achieved
    final_bounds = history.fuel_bounds_history[-1]
    final_range = final_bounds[1] - final_bounds[0]
    validation_results['converged'] = final_range < CONVERGENCE_TOLERANCE_KG

    # Check 2: Reasonable iterations
    num_iterations = len(history.iterations)
    validation_results['iteration_count_ok'] = num_iterations <= MAX_ITERATIONS

    # Check 3: Monotonic bounds reduction
    ranges = [bounds[1] - bounds[0] for bounds in history.fuel_bounds_history]
    is_monotonic = all(ranges[i] >= ranges[i+1] for i in range(len(ranges)-1))
    validation_results['monotonic_convergence'] = is_monotonic

    # Check 4: Best result quality
    best_deficit = min(abs(it.fuel_deficit_kg) for it in history.iterations)
    validation_results['deficit_acceptable'] = best_deficit < 50.0 # kg

    # Print validation summary
    print("\n" + "="*80)
    print("CONVERGENCE VALIDATION")
    print("="*80)
    for check, result in validation_results.items():
        status = "✓ PASS" if result else "✗ FAIL"
        print(f"{check}: {status}")
    print("="*80 + "\n")

    return validation_results

```

8.2 Physical Consistency Validation

Mass Conservation:

$$m_{initial} - m_{final} = F_{consumed}$$

Implementation:

```
def validate_mass_conservation(iteration_result: MissionIterationResults) -> bool:
    """
    Validate mass conservation throughout mission.

    Checks:
        m_initial = m_final + F_total

    Returns:
        True if mass is conserved within tolerance
    """
    m_initial = iteration_result.initial_mass_kg
    m_final = iteration_result.final_weight_kg
    F_consumed = iteration_result.fuel_consumed_kg

    mass_balance = m_initial - (m_final + F_consumed)
    tolerance = 0.1 # kg

    is_valid = abs(mass_balance) < tolerance

    if not is_valid:
        print(f"[WARNING] Mass conservation violated: {mass_balance:.2f} kg discrepancy")
        print(f"  Initial mass: {m_initial:.1f} kg")
        print(f"  Final mass: {m_final:.1f} kg")
        print(f"  Fuel consumed: {F_consumed:.1f} kg")
        print(f"  Expected final: {m_initial - F_consumed:.1f} kg")

    return is_valid
```

Energy Conservation:

$$E_{initial} - E_{final} \approx E_{fuel,chemical} - E_{dissipated}$$

Performance Envelope:

- All phases remain within flight envelope
- No stall conditions encountered
- MMO (Maximum Operating Mach) not exceeded
- Engine limits respected

8.3 Performance Monitoring

Key Metrics to Monitor:

1. **Fuel Breakdown:** Typical distribution
 - Climb: 15-20% of total
 - Cruise: 70-80% of total
 - Descent: 1-5% of total
2. **Convergence Behavior:**
 - Bounds should reduce monotonically
 - No iteration failures after first iteration
 - Deficit should approach zero
3. **Physical Realism:**
 - Final fuel capacity reasonable for aircraft class
 - Mission duration consistent with distance/speed
 - Fuel consumption rates within expected ranges

Monitoring Function:

```

def monitor_optimization_quality(history: ConvergenceHistory) -> Dict[str, Any]:
    """
    Monitor and report optimization quality metrics.

    Returns:
        Dictionary of quality metrics
    """
    metrics = {}

    # Convergence metrics
    final_bounds = history.fuel_bounds_history[-1]
    metrics['final_range_kg'] = final_bounds[1] - final_bounds[0]
    metrics['num_iterations'] = len(history.iterations)

    # Best result analysis
    best_idx = np.argmin([abs(it.fuel_deficit_kg) for it in history.iterations])
    best_result = history.iterations[best_idx]
    metrics['best_deficit_kg'] = best_result.fuel_deficit_kg
    metrics['best_fuel_kg'] = best_result.initial_fuel_kg

    # Fuel breakdown
    metrics['climb_fraction'] = best_result.climb_fuel_kg / best_result.fuel_consumed_kg
    metrics['cruise_fraction'] = best_result.cruise_fuel_kg / best_result.fuel_consumed_kg
    metrics['descent_fraction'] = best_result.descent_fuel_kg / best_result.fuel_consumed_kg

    # Print monitoring report
    print("\n" + "="*80)
    print("OPTIMIZATION QUALITY METRICS")
    print("="*80)
    print(f"Convergence:")
    print(f"  Final range: {metrics['final_range_kg']:.1f} kg")
    print(f"  Iterations: {metrics['num_iterations']}")
    print(f"Best Result:")
    print(f"  Fuel: {metrics['best_fuel_kg']:.1f} kg")
    print(f"  Deficit: {metrics['best_deficit_kg']:+.1f} kg")
    print(f"Fuel Breakdown:")
    print(f"  Climb: {metrics['climb_fraction']*100:.1f}%")
    print(f"  Cruise: {metrics['cruise_fraction']*100:.1f}%")
    print(f"  Descent: {metrics['descent_fraction']*100:.1f}%")

```

```
print("="*80 + "\n")

return metrics
```

9) Convergence Analysis and Characteristics

9.1 Bisection Convergence Characteristics

Theoretical Properties:

1. **Guaranteed Convergence:** Always converges if solution exists in initial bracket
2. **Monotonic Reduction:** Search range halves each iteration
3. **Predictable Iterations:** Known from $\log_2(\text{initial_range} / \text{tolerance})$
4. **No Parameter Tuning:** No damping factors or acceleration parameters needed

Convergence Rate Comparison:

Method	Rate	Formula	Typical Iterations
Bisection	Linear in range	$\text{Range}_n = \text{Range}_0 / 2^n$	10-12
Fixed-Point (no Aitken)	Linear in error	$e_n = (1-\omega)^n e_0$	15-25
Fixed-Point (with Aitken)	Quasi-quadratic	$e_n = O(e_{n-1}^2)$ near root	8-15 (if stable)
Newton-Raphson	Quadratic	$e_n = O(e_{n-1}^2)$	4-6 (if gradient available)

Bisection Advantages:

- ✓ No divergence possible
- ✓ No oscillations
- ✓ Robust to discretization effects from DP grids

- ✓ Simple implementation
- ✓ No parameter tuning required

9.2 Typical Convergence Sequence

Example: Standard Mission

Initial conditions:

- $F_{low} = 1,000$ kg
- $F_{high} = 10,500$ kg
- Tolerance = 10 kg

Iter	F_low	F_high	F_mid	F_consumed	Deficit	Range	Action
1	1,000	10,500	5,750	4,850	-900	9,500	↓ Upper
2	1,000	5,750	3,375	3,890	+515	4,750	↑ Lower
3	3,375	5,750	4,563	4,420	-143	2,375	↓ Upper
4	3,375	4,563	3,969	4,025	+56	1,188	↑ Lower
5	3,969	4,563	4,266	4,248	-18	594	↓ Upper
6	3,969	4,266	4,118	4,135	+17	297	↑ Lower
7	4,118	4,266	4,192	4,191	-1	148	↓ Upper
8	4,118	4,192	4,155	4,163	+8	74	↑ Lower
9	4,155	4,192	4,173	4,177	+4	37	↑ Lower
10	4,173	4,192	4,183	4,184	+1	19	↑ Lower
11	4,183	4,192	4,187	4,188	+1	9	✓ Converged

Convergence Pattern:

- Range reduces by factor of 2 each iteration
- Deficit oscillates in sign as brackets tighten

- Magnitude of deficit decreases as solution approached
- Final deficit typically within ± 5 kg of equilibrium

9.3 Handling Edge Cases

Case 1: Mission Failure (Insufficient Fuel)

```
try:
    iteration_result = run_single_mission_iteration(initial_fuel_kg=fuel_mid, ...)
except RuntimeError as e:
    # Mission failed - insufficient fuel
    print(f"[ERROR] Mission failed: {str(e)}")
    print(f"[BISECTION] Mission failure indicates insufficient fuel")
    fuel_low = fuel_mid # Increase lower bound
    continue
```

Case 2: Non-Convergence (Max Iterations Reached)

```
if iteration_count >= MAX_ITERATIONS:
    print(f"[WARNING] Reached MAX_ITERATIONS without full convergence")
    print(f"Final range: {fuel_high - fuel_low:.1f} kg (tolerance: {CONVERGENCE_TOLERANCE_KG:.1f})")
    print(f"Using best result from iteration {best_result.iteration}")
```

Case 3: Initial Bracket Invalid

```
# First iteration with F_high fails
if iteration_count == 1:
    raise RuntimeError(
        f"Mission infeasible even with high fuel estimate ({fuel_high:.1f} kg). "
        f"Check mission parameters or increase INITIAL_FUEL_HIGH_KG."
    )
```

9.4 Discretization Effects

DP Grid Sensitivity:

Dynamic programming uses discrete grids:

- Altitude: N_ALTITUDE_STEPS (50-100)
- Mach: N_MACH_SAMPLES (61-101)
- Lever: N_LEVER_SAMPLES (50-100)

Impact on Convergence:

Small mass changes can cause optimizer to select different discrete states, leading to:

- Small trajectory variations
- Non-smooth fuel response (± 10 -50 kg jumps)
- Local fuel consumption variations

Why Bisection Handles This Well:

- Brackets always tighten monotonically regardless of small variations
- Final result guaranteed within tolerance band
- No oscillation or divergence from discretization noise

Mitigation Strategies:

1. Set tolerance (10 kg) larger than typical discretization noise (± 5 kg)
2. Use "best result" selection (smallest |deficit|) rather than last iteration
3. Apply safety buffer (5%) to account for variations

10) Advanced Topics and Extensions

10.1 Alternative Optimization Methods

Secant Method:

Modification of bisection using secant approximation for faster convergence:

```
def secant_update(F_1, F_2, Deficit_1, Deficit_2):
    """
    Secant method update for superlinear convergence.

    Instead of midpoint, use linear interpolation:
        F_next = F_2 - Deficit_2 * (F_2 - F_1) / (Deficit_2 - Deficit_1)
    """
    denominator = Deficit_2 - Deficit_1
    if abs(denominator) > 1e-6:
        F_next = F_2 - Deficit_2 * (F_2 - F_1) / denominator
        return F_next
    else:
        # Fall back to bisection
        return (F_1 + F_2) / 2.0
```

Advantages:

- Superlinear convergence (faster than bisection)
- Still robust (no derivatives needed)

Disadvantages:

- Can exit bracket (requires safeguarding)
- More complex than pure bisection

10.2 Hybrid Bisection-Secant

Safeguarded Secant Method:

```

def hybrid_bisection_secant(F_low, F_high, Deficit_low, Deficit_high):
    """
    Hybrid method: Use secant with bisection safeguard.

    Process:
    1. Compute secant estimate
    2. Check if within bracket
    3. If outside: use bisection
    4. If inside: use secant
    """
    # Secant estimate
    F_secant = secant_update(F_low, F_high, Deficit_low, Deficit_high)

    # Safeguard: ensure within bracket
    if F_low < F_secant < F_high:
        # Use secant
        F_next = F_secant
        print(f"[HYBRID] Using secant estimate: {F_next:.1f} kg")
    else:
        # Fall back to bisection
        F_next = (F_low + F_high) / 2.0
        print(f"[HYBRID] Secant out of bounds, using bisection: {F_next:.1f} kg")

    return F_next

```

10.3 Multi-Objective Optimization

Objective: Minimize fuel while constraining mission time or other metrics

```

def optimize_fuel_with_time_constraint(
    aero, eng, M_grid, H_plot,
    max_mission_time_hours: float = 4.0
):
    """
    Bisection optimization with mission time constraint.

    Modified algorithm:
    - Standard bisection for fuel
    - Check time constraint after each iteration
    - If time exceeded: increase cruise Mach or adjust climb profile
    """
    # Standard bisection loop
    while not converged:
        result = run_single_mission_iteration(fuel_mid, ...)

        # Check time constraint
        mission_time_hours = result.total_time_s / 3600
        if mission_time_hours > max_mission_time_hours:
            print(f"[WARNING] Time constraint violated: {mission_time_hours:.2f} h > {max_mission_time_hours:.2f} h")
            # Adjust mission profile (increase cruise speed, etc.)

        # Continue bisection...

```

10.4 Uncertainty Quantification

Approach: Run bisection with perturbed parameters to assess fuel uncertainty

```

def optimize_with_uncertainty_analysis(
    aero, eng, M_grid, H_plot,
    n_samples: int = 10,
    mass_uncertainty_kg: float = 100.0,
    wind_uncertainty_mps: float = 5.0
):
    """
    Monte Carlo analysis of fuel optimization uncertainty.

    Process:
    1. Run bisection n_samples times
    2. Perturb parameters each run (mass, wind, drag)
    3. Analyze distribution of optimized fuel
    """
    optimized_fuels = []

    for i in range(n_samples):
        # Perturb parameters
        W_OE_perturbed = W_OE_KG + np.random.normal(0, mass_uncertainty_kg)
        wind_perturbed = np.random.normal(0, wind_uncertainty_mps)

        # Run optimization with perturbed parameters
        result, history = optimize_fuel_capacity(aero, eng, M_grid, H_plot)

        optimized_fuels.append(result.fuel_consumed_kg * 1.05)

    # Analyze results
    fuel_mean = np.mean(optimized_fuels)
    fuel_std = np.std(optimized_fuels)
    fuel_95_percentile = np.percentile(optimized_fuels, 95)

    print(f"Fuel uncertainty analysis:")
    print(f"  Mean: {fuel_mean:.1f} kg")
    print(f"  Std: {fuel_std:.1f} kg")
    print(f"  95th percentile: {fuel_95_percentile:.1f} kg")

    return {
        'mean': fuel_mean,
        'std': fuel_std,
    }

```

```
    'percentile_95': fuel_95_percentile,  
    'samples': optimized_fuels  
}
```

10.5 Adaptive Grid Refinement

Concept: Refine DP grids during optimization for better accuracy

```

def optimize_with_adaptive_refinement(aero, eng, M_grid, H_plot):
    """
    Bisection optimization with adaptive grid refinement.

    Process:
    1. Start with coarse DP grids (fast)
    2. Run bisection until near convergence
    3. Refine grids (altitude, Mach, lever)
    4. Continue bisection with refined grids
    """
    # Phase 1: Coarse grid optimization
    print("[PHASE 1] Coarse grid optimization...")
    N_ALT_COARSE = 30
    N_MACH_COARSE = 41
    N_LEVER_COARSE = 25

    result_coarse, history_coarse = optimize_fuel_capacity(
        aero, eng, M_grid, H_plot,
        lever_samples=N_LEVER_COARSE
    )

    # Phase 2: Fine grid refinement near solution
    print("[PHASE 2] Fine grid refinement...")
    N_ALT_FINE = 100
    N_MACH_FINE = 101
    N_LEVER_FINE = 50

    # Narrow bounds around coarse solution
    fuel_coarse = result_coarse.fuel_consumed_kg * 1.05
    F_low_refined = fuel_coarse * 0.95
    F_high_refined = fuel_coarse * 1.05

    result_fine, history_fine = optimize_fuel_capacity(
        aero, eng, M_grid, H_plot,
        lever_samples=N_LEVER_FINE
    )

    return result_fine, (history_coarse, history_fine)

```

11) Summary and Conclusion

11.1 Key Capabilities

The fuel capacity optimization system provides:

- ✓ **Minimum Fuel Determination:** Systematic optimization to find minimum required fuel using bisection
- ✓ **Guaranteed Convergence:** Monotonic, robust convergence without oscillations
- ✓ **Dynamic Mass Tracking:** Accurate mass evolution throughout mission
- ✓ **Fuel Deficit Tracking:** Clear monitoring of consumed vs. available fuel
- ✓ **Safety Integration:** Automatic safety buffer application
- ✓ **Robust Operation:** Error handling and recovery mechanisms

11.2 System Strengths

Mathematical Rigor:

- Implementation of bisection method with guaranteed monotonic convergence
- Absolute tolerance criterion (10 kg) for precise fuel determination
- Bounded search with predictable iteration count
- No parameter tuning required (damping-free)

Physical Accuracy:

- Dynamic mass evolution with fuel burn
- Weight-dependent aerodynamic calculations
- Coupled climb-cruise-descent optimization
- Mass and energy conservation throughout mission

Software Quality:

- Modular design with clear separation of concerns
- Comprehensive error handling and validation
- Fuel deficit tracking for transparent optimization progress
- Convergence history management for analysis and diagnostics

Advantages Over Previous Approach:

- ✓ No oscillations or divergence
- ✓ Predictable convergence behavior
- ✓ Simpler implementation (no damping parameters)
- ✓ More robust to DP discretization effects
- ✓ Guaranteed convergence if solution exists

11.3 Recommended Workflow

1. **Configure Mission** in `mission_config.py`
2. **Execute Optimizer:** `python main_fuel_optimizer.py`
3. **Review Convergence:** Check iteration history and bounds progression
4. **Analyze Results:** Examine optimized fuel and deficit values
5. **Validate Physics:** Verify mass conservation and performance envelope
6. **Document Results:** Use convergence plots for reporting

11.4 Best Practices

Parameter Selection:

- Convergence tolerance: 10 kg (typical, ~0.07% for 15,000 kg fuel)
- Safety buffer: 5% (industry standard)
- Max iterations: 20 (sufficient for typical range)
- Initial bounds: [1,000 kg, MAX_FUEL_KG] (conservative bracket)

Validation:

- Always check final convergence range < tolerance
- Monitor fuel deficit progression toward zero
- Verify mass conservation throughout mission
- Check performance metrics for physical realism
- Review iteration history for monotonic bounds reduction

Troubleshooting:

- Mission failure at F_{high} → Check mission feasibility, increase INITIAL_FUEL_HIGH_KG
- Non-convergence → Increase MAX_ITERATIONS or check tolerance

- Large deficit oscillations → Check DP grid resolution
- Physical anomalies → Verify aerodynamics/engine data validity

11.5 Typical Results

Performance Metrics:

- Convergence: 9-11 iterations typical
- Final range: < 10 kg
- Best deficit: < 5 kg (within 0.1% of equilibrium)
- Computation time: ~10-15 minutes per iteration
- Total optimization time: ~2-3 hours

Fuel Savings:

- Typical: 30-40% reduction from MAX_FUEL_KG
 - Example: 10,500 kg → 4,500 kg optimized (57% savings)
 - With safety buffer: 4,500 kg → 4,725 kg (5% added)
-

Document Version: 3.0 (Bisection Implementation)

Last Updated: November 2025

Status: Production-ready

Maintained by: Mission Analysis System

Key Changes from Version 2.0:

- Replaced fixed-point iteration with bisection method
- Removed Aitken acceleration (no longer needed)
- Updated convergence criterion to absolute tolerance
- Added fuel deficit tracking
- Removed performance calculator (simplified data structure)
- Updated all documentation to reflect bisection approach